

INFORME DE MACHINE LEARNING

Caso de Estudio: Talent Retention

People Analytics aplicado a Recursos Humanos



Elemento	Descripción
Metodología	CRISP-DM (sin fase de despliegue)
Sector	Recursos Humanos
Rol simulado	People Analytics
Regresión	MonthlyIncome
Clasificación	Attrition (fuga de talento)
Dataset	IBM HR Analytics Employee Attrition & Performance
Registros / variables	1.470 registros / 35 variables originales
Autor registrado en notebook	Alan Maldonado, Juan Claros, Brayan Apaza

Resumen ejecutivo

El presente informe documenta el desarrollo de un proyecto de Machine Learning para el caso de estudio Talent Retention, utilizando la metodología CRISP-DM. El análisis se desarrolla desde la perspectiva de un equipo de People Analytics de Recursos Humanos y aborda dos problemas supervisados sobre el mismo conjunto de datos: una regresión para estimar MonthlyIncome y una clasificación binaria para anticipar Attrition.

El notebook parte de 1.470 empleados y 35 variables. Durante la exploración se comprobó que no existen valores nulos y se identificó un fuerte desbalance en Attrition: 83,88% de los registros corresponden a empleados que no abandonaron la organización y 16,12% a empleados que sí lo hicieron. También se detectó una elevada correlación entre JobLevel, MonthlyIncome y variables de antigüedad, lo que justificó la comparación de Regresión Lineal, Ridge y Lasso.

Para la regresión, la Regresión Lineal obtuvo en el conjunto de prueba $MAE = 895,372$, $RMSE = 1.169,208$ y $R^2 = 0,937$, superando ampliamente al baseline de mediana. Para la clasificación, la regresión logística ajustada mediante validación cruzada obtuvo en test $ROC-AUC = 0,812$ y $PR-AUC = 0,589$. Debido al mayor costo de un falso negativo, se seleccionó un umbral operativo de 0,55, con el cual se obtuvieron 31 verdaderos positivos y 16 falsos negativos en el conjunto de prueba.

Mapa del proceso CRISP-DM

Fase CRISP-DM	Aplicación en el proyecto	Estado
1. Comprensión del negocio	Definición de objetivos de People Analytics, costos FN/FP y targets.	Completada
2. Comprensión de los datos	Carga, calidad, EDA, distribución de targets, correlaciones y leakage.	Completada
3. Preparación de los datos	Limpieza, codificación, selección de variables, split y pipelines.	Completada
4. Modelado	Baselines, Regresión Lineal, Ridge, Lasso y Regresión Logística.	Completada
5. Evaluación	Métricas, validación cruzada, curvas, residuos, umbral operativo e interpretación.	Completada
6. Despliegue	No se implementa en esta etapa del proyecto.	Pendiente

1. Comprensión del negocio

La organización plantea un problema típico de People Analytics: comprender qué factores se relacionan con la remuneración de sus empleados y, simultáneamente, anticipar posibles fugas de talento. El objetivo no es automatizar decisiones de RRHH, sino proporcionar evidencia cuantitativa que ayude a priorizar revisiones salariales y conversaciones de retención.

1.1 Objetivos de negocio

- Analizar los factores asociados al ingreso mensual de los empleados y detectar desviaciones importantes entre el salario real y el esperado.
- Identificar empleados con mayor probabilidad estimada de abandonar la organización.
- Reducir el riesgo de perder talento sin anticipación, priorizando Recall y PR-AUC en la clasificación.
- Generar recomendaciones accionables para Compensación & Beneficios y para los responsables de gestión de personas.
- Mantener supervisión humana y criterios éticos, especialmente frente a variables demográficas sensibles.

1.2 Definición de los targets

Tarea	Target	Tipo	Pregunta de negocio
Regresión	MonthlyIncome	Continuo	¿Qué factores explican el salario mensual y qué empleados presentan desviaciones importantes respecto a perfiles comparables?
Clasificación	Attrition	Binario	¿Qué empleados presentan mayor riesgo estimado de renunciar?

1.3 Costos de error y criterio de éxito

En la clasificación existe una asimetría de costos. Un falso positivo implica realizar una intervención de retención sobre una persona que probablemente no iba a renunciar. Un falso negativo implica no detectar a una persona que finalmente abandona la empresa, con posibles costos de reemplazo, onboarding y pérdida de conocimiento.

Resultado	Interpretación de negocio	Prioridad
Verdadero positivo (TP)	Se identifica correctamente una fuga potencial.	Deseable
Verdadero negativo (TN)	Se identifica correctamente a quien permanece.	Deseable
Falso positivo (FP)	Intervención preventiva innecesaria.	Costo relativamente bajo
Falso negativo (FN)	Fuga no detectada.	Costo alto

Por este motivo, el notebook evita utilizar Accuracy como criterio principal de éxito. Se priorizan Recall, Precision, ROC-AUC y especialmente PR-AUC, además de una selección explícita del umbral según un costo relativo de FN = 5 y FP = 1.

Código/documentación del contexto y objetivos del negocio

```
1 ## 1. Contexto del problema
2
3 ## 1.1 Descripción de negocio
4 "Objetivo (Business):"==> reducir la compensación a beneficios identificando qué variables explican el salario y detectar qué empleados cuyo salario real se aleja mucho del esperado según su perfil (antigüedad, nivel, rol) - incluso para revisiones de equidad salarial.
5 "Justificación (Business):"==> ayudar a los gerentes de línea priorizando conversaciones de retención, anticipando qué empleados tienen mayor probabilidad de renunciar."
6
7 ## 1.2 Contexto de negocio: Palcos Negativos vs. Palcos Positivos
8
9 | Predicción se queda | Predicción se va |
10 |-----|-----|
11 "Real" se queda" | Correcta (TN) | "Palco Positivo (PP)"==> costo de una intervención de retención (tiempo de gerencia, incentivos) sobre alguien que no iba a renunciar. Costo relativamente bajo. |
12 "Real" se va" | "Palco Negativo (PN)"==> se pierde talento sin retención - costo de reemplazo (recruitment) + onboarding + curva de aprendizaje. Si no se va el salario actual del gerente, más pérdida de conocimiento del equipo. Costo alto. | Correcta (FP) |
13
14 "Decisión de negocio"==> el costo de un "Palco Negativo" es sustancialmente mayor que el de un "Palco Positivo". Por lo tanto priorizamos "Recall" (detectar la mayor cantidad posible de fugas reales) sin descuidar una Precisión razonable, y "no usamos Accuracy" como métrica principal porque el target está desbalanceado (Sección 2.4).
15
16 ## 1.3 Descripción de variables
17 "Predictores" (features manual del empleado (ID)): variable continua = "regresión"
18 "Predictor" es el empleado (nombre) y "Y" es "se queda" o "se va". Se clasifican a binario 0/1 "Fuga", 1="se queda" = "clasificación".
19
20 ## 1.4 El "dato" del data loader es IDNN
21
22 Antes de hacer feature engineering los datos se complementan (cada uno con su evidencia):
23
24 "Fuga de información (feature leakage)": existe alguna variable que en la realidad "no" estaría disponible en el momento T-0" (al instante en que NNNN necesita predecir), por reflejar información posterior a la fuga o a la decisión salarial? Este análisis se documenta como una "tabla explícita de variables perdidas vs. no perdidas" en la Sección 2.7, con el motivo de cada una.
25
26 "Columnas constantes (irrelevantes)": variables sin variación que son identificadores, verificadas con evidencia (varían a lo largo de la Sección 3.1) antes de eliminarlas: EmployeeID (identificador único), EmployeeName, Gender (masculino), Over30 (constante).
27
28 ## 1.5 Resumen de la conclusión: en este dataset "no encontramos variables con fuga temporal" - es una fotografía histórica de empleados antes/retirados, y todos los variables relevantes (satisfacción, desempeño, antigüedad, etc.) son información disponible "antes" de la fuga o de la decisión salarial. Si encontramos columnas constantes/identificadores, que se eliminan en la Sección 3.1.
```

2. Comprensión de los datos

El dataset utilizado es IBM HR Analytics Employee Attrition & Performance, descargado desde Kaggle mediante la API de Kaggle dentro del notebook. El conjunto contiene 1.470 observaciones y 35 variables, combinando variables numéricas y categóricas de carácter laboral y demográfico.

Característica	Resultado
Filas	1.470
Columnas originales	35
Tipos numéricos	26
Tipos categóricos (object)	9
Valores nulos	0
Target regresión	MonthlyIncome
Target clasificación	Attrition

Carga del dataset mediante Kaggle API

```

1 KAGGLE_API_URL = (
2     "https://www.kaggle.com/api/v1/datasets/download/"
3     "pavansubhasht/ibm-hr-analytics-attrition-dataset"
4 )
5 URL_RESPALDO = (
6     "https://raw.githubusercontent.com/nelson-wu/employee-attrition-ml/"
7     "master/WA_Fn-UseC_HR-Employee-Attrition.csv"
8 )
9
10 def cargar_dataset():
11     # Descarga desde la Kaggle API; si falla, usa un enlace de respaldo
12     # verificado como identico byte a byte al CSV oficial de Kaggle.
13     try:
14         peticion = urllib.request.Request(KAGGLE_API_URL, headers={"User-Agent": "Mozilla/5.0"})
15         with urllib.request.urlopen(peticion, timeout=30) as respuesta:
16             contenido_zip = respuesta.read()
17             with zipfile.ZipFile(io.BytesIO(contenido_zip)) as zf:
18                 nombre_csv = [n for n in zf.namelist() if n.endswith(".csv")][0]
19                 with zf.open(nombre_csv) as f:
20                     df_cargado = pd.read_csv(f)
21             print("Dataset descargado desde la Kaggle API (fuente oficial del caso).")
22             return df_cargado
23     except Exception as error:
24         print(f"No se pudo usar la Kaggle API ({type(error).__name__}: {error}).")
25         print("Se usa el enlace de respaldo (mismo dataset, verificado identico byte a byte).")
26         return pd.read_csv(URL_RESPALDO)
27
28 df = cargar_dataset()
29 print("Forma del dataset:", df.shape)
30 df.head()

```

El notebook incorpora además una comprobación de forma y de existencia de los targets, lo que funciona como control de integridad previo a continuar con el análisis.

2.1 Calidad de los datos

La revisión de tipos y valores ausentes mostró 26 columnas numéricas y 9 categóricas, sin valores nulos en ninguna de las 1.470 observaciones. Aunque no fue necesario imputar datos faltantes, el pipeline mantiene imputación defensiva mediante mediana para variables numéricas y moda para categóricas, de forma que el flujo sea más robusto ante datos futuros.

Comprobación de tipos y valores nulos

```

1 display(df.dtypes.value_counts())
2 print("\nTotal de valores nulos en el dataset:", int(df.isna().sum().sum()))
3 df.describe().T.head(10)

```

2.2 Distribución de MonthlyIncome

MonthlyIncome presenta una distribución sesgada hacia la derecha. La media es de 6.502,93, mientras que la mediana es de 4.919 y el máximo alcanza 19.999. Esta diferencia refleja la existencia de empleados con niveles jerárquicos y remuneraciones considerablemente superiores.

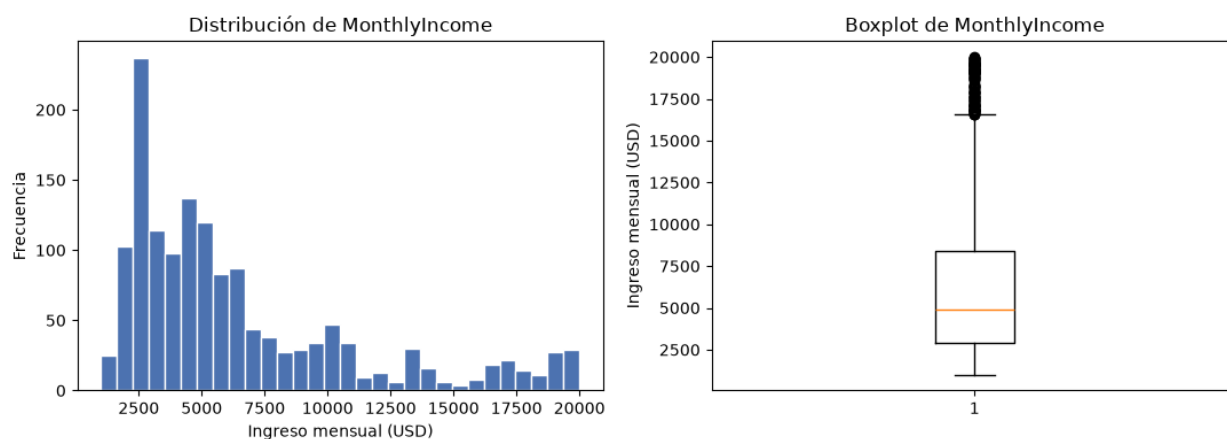


Figura 1. Histograma y diagrama de caja de MonthlyIncome.

2.3 Distribución de Attrition

De los 1.470 empleados, 1.233 no abandonaron la organización y 237 sí lo hicieron. Esto representa 83,88% de clase No frente a 16,12% de clase Yes. El desbalance es relevante porque un clasificador que siempre predijera No alcanzaría aproximadamente 84% de Accuracy sin detectar ninguna fuga.

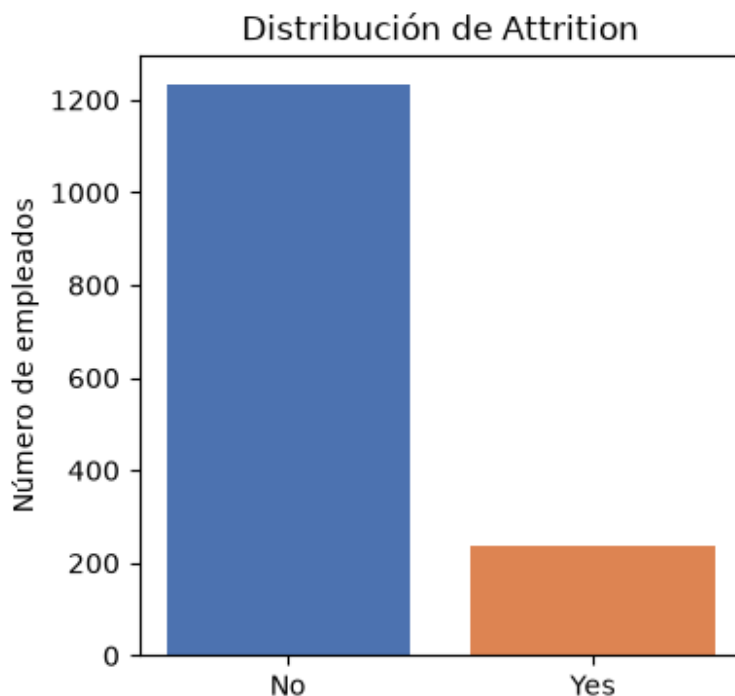


Figura 2. Distribución de Attrition: 83,88% No y 16,12% Yes.

Cálculo de frecuencia y proporción de Attrition

```

1 conteo = df["Attrition"].value_counts()
2 proporcion = df["Attrition"].value_counts(normalize=True) * 100
3
4 print("Conteo:\n", conteo)
5 print("\nProporción (%):\n", proporcion.round(2))
6
7 plt.figure(figsize=(4, 4))
8 plt.bar(conteo.index, conteo.values, color=["#4C72B0", "#DD8452"])
9 plt.title("Distribución de Attrition")
10 plt.ylabel("Número de empleados")
11 plt.show()

```

2.4 Correlaciones y colinealidad

El análisis de correlaciones muestra relaciones especialmente fuertes entre JobLevel y MonthlyIncome ($r = 0,9503$), JobLevel y TotalWorkingYears ($r = 0,7822$), MonthlyIncome y TotalWorkingYears ($r = 0,7729$), y diferentes medidas de antigüedad. Esta estructura es coherente con la lógica salarial de una organización: el nivel jerárquico y la experiencia se relacionan estrechamente con el ingreso.

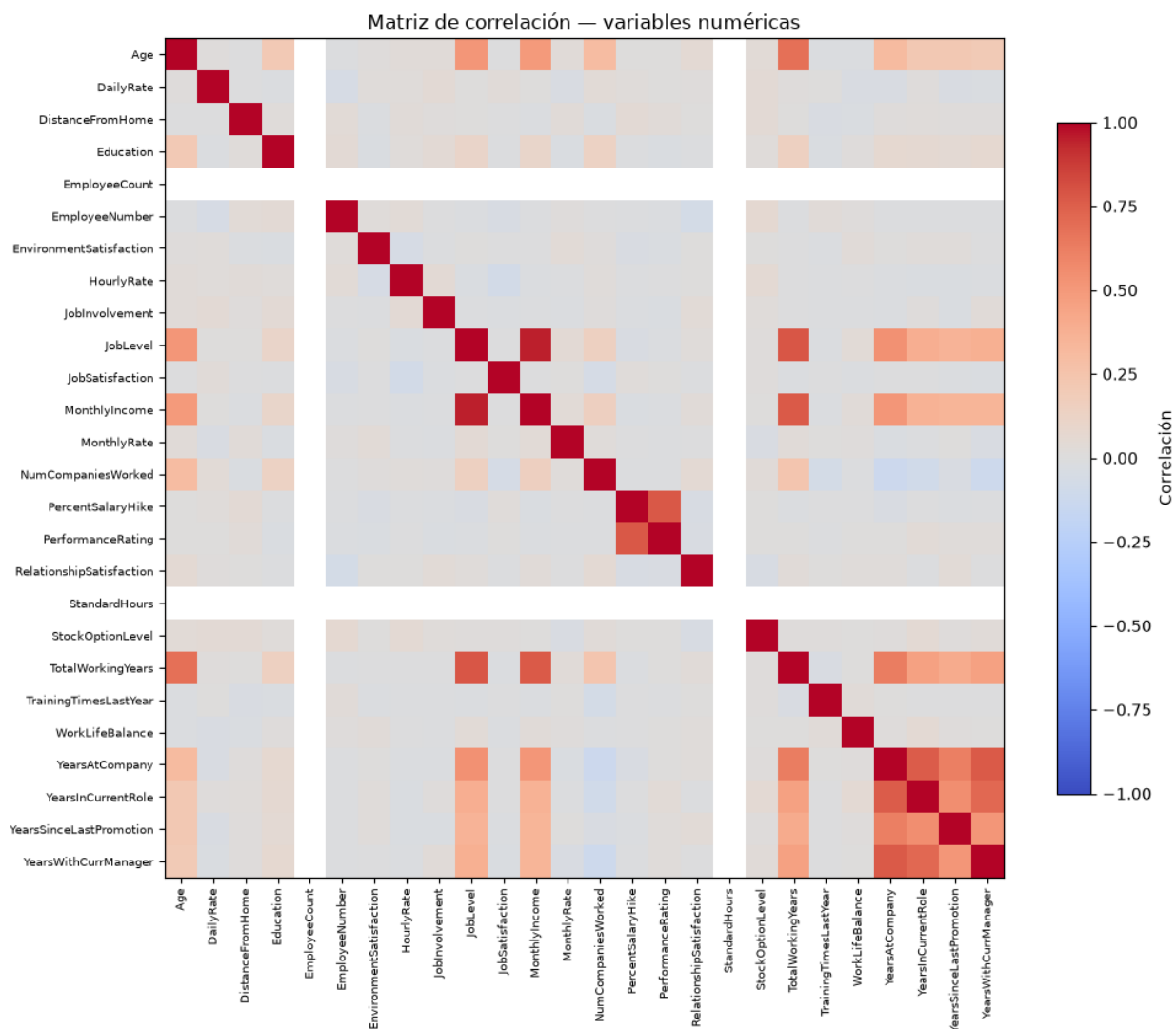


Figura 3. Matriz de correlaciones entre variables numéricas.

2.5 Relaciones con la fuga de talento

El EDA del notebook compara Attrition con variables de negocio como horas extra, salario y balance vida-trabajo. Se observa una asociación importante entre OverTime = Yes y una mayor frecuencia de fuga. Estas señales se utilizan posteriormente para interpretar el modelo de clasificación, sin interpretarlas como causalidad.

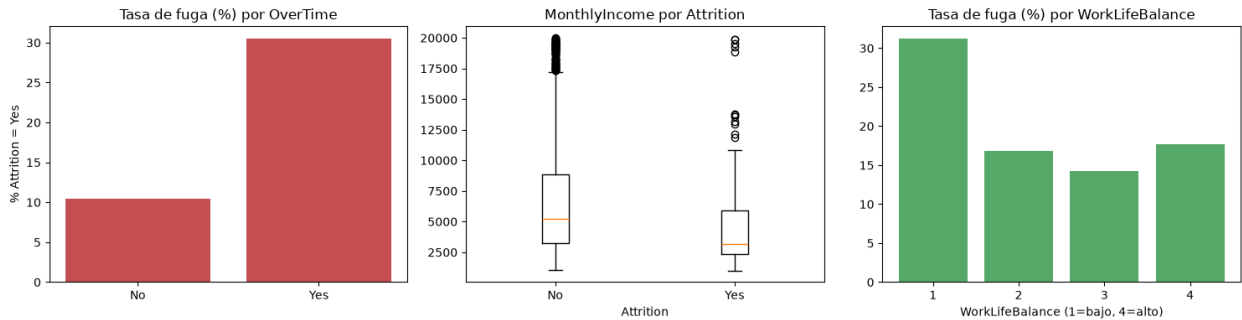


Figura 4. Comparación de Attrition frente a variables clave de negocio.

2.6 Análisis de fuga de información (data leakage)

Antes del modelado se revisaron las 35 variables originales para determinar si podían utilizarse legítimamente en el momento de decisión T-0. El notebook concluye que no se identificaron variables con fuga temporal. Sin embargo, sí se detectaron cuatro variables que debían excluirse por ser identificadores o constantes: EmployeeNumber, EmployeeCount, StandardHours y Over18.

Variable	Tratamiento	Motivo
EmployeeNumber	Excluir	Identificador único; no aporta información generalizable.
EmployeeCount	Excluir	Constante: un único valor.
StandardHours	Excluir	Constante: un único valor.
Over18	Excluir	Constante en el conjunto.

Código de diagnóstico de leakage y clasificación de variables

```
1 # Diagnóstico de fuga de información (leakage) para las 35 columnas originales.
2 # "Permitida": info disponible en T-0 (antes de la fuga / de fijar el salario).
3 # "Excluida": no aporta señal real (constante/identificador) o es el propio target.
4 diagnostico_leakage = {
5     "Age": ("Permitida", "Demográfica estable, disponible en T-0."),
6     "Attrition": ("Target (clasificación)", "Variable objetivo cruda: no se usa como feature."),
7     "BusinessTravel": ("Permitida", "Condición laboral conocida en T-0 (variable ordinal, ver Sección 3.6)."),
8     "DailyRate": ("Permitida", "Dato administrativo vigente en T-0."),
9     "Department": ("Permitida", "Estructura organizacional conocida en T-0."),
10    "DistanceFromHome": ("Permitida", "Dato demográfico estable."),
11    "Education": ("Permitida", "Nivel educativo, fijo históricamente."),
12    "EducationField": ("Permitida", "Categoría nominal conocida en T-0."),
13    "EmployeeCount": ("Excluida", "Constante (varianza 0, nunique=1) - sin señal (evidencia en Sección 3.1)."),
14    "EmployeeNumber": ("Excluida", "Identificador único (nunique = n filas) - riesgo de sobreajuste, sin poder predictivo (evidencia en Sección 3.1)."),
15    "EnvironmentSatisfaction": ("Permitida", "Encuesta de clima vigente, anterior a la fuga."),
16    "Gender": ("Permitida (sensible)", "Variable protegida: se usa pero se audita su peso en la Sección 6."),
17    "HourlyRate": ("Permitida", "Dato administrativo vigente en T-0."),
18    "JobInvolvement": ("Permitida", "Evaluación vigente, anterior a la fuga."),
19    "JobLevel": ("Permitida", "Estructura jerárquica conocida en T-0."),
20    "JobRole": ("Permitida", "Categoría nominal conocida en T-0."),
21    "JobSatisfaction": ("Permitida", "Encuesta vigente, anterior a la fuga (no es resultado posterior)."),
22    "MaritalStatus": ("Permitida (sensible)", "Variable protegida: se usa pero se audita su peso en la Sección 6."),
23    "MonthlyIncome": ("Target (regresión) / Permitida en clasificación", "Target de la Tarea 1: como feature de Attrition es válida (vigente en T-0), pero se excluye de las features de regresión por ser su propio target."),
24    "MonthlyRate": ("Permitida", "Dato administrativo vigente en T-0 (distinto de MonthlyIncome)."),
25    "NumCompaniesWorked": ("Permitida", "Histórico laboral previo, conocido en T-0."),
26    "Over18": ("Excluida", "Constante (un único valor 'Y') - sin señal (evidencia en Sección 3.1)."),
27    "OverTime": ("Permitida", "Condición laboral vigente, conocida en T-0."),
28    "PercentSalaryHike": ("Permitida", "Del ciclo de compensación más reciente, anterior a la fuga."),
29    "PerformanceRating": ("Permitida", "Evaluación más reciente, anterior a la decisión de fuga."),
30    "RelationshipSatisfaction": ("Permitida", "Encuesta vigente, anterior a la fuga."),
31    "StandardHours": ("Excluida", "Constante (un único valor, 80) - sin señal (evidencia en Sección 3.1)."),
32    "StockOptionLevel": ("Permitida", "Beneficio vigente, conocido en T-0."),
33    "TotalWorkingYears": ("Permitida", "Histórico laboral acumulado, conocido en T-0."),
34    "TrainingTimesLastYear": ("Permitida", "Histórico del año anterior, conocido en T-0."),
35    "WorkLifeBalance": ("Permitida", "Encuesta vigente, anterior a la fuga."),
36    "YearsAtCompany": ("Permitida", "Antigüedad conocida en T-0."),
37    "YearsInCurrentRole": ("Permitida", "Antigüedad en el rol, conocida en T-0."),
38    "YearsSinceLastPromotion": ("Permitida", "Histórico conocido en T-0."),
39    "YearsWithCurrManager": ("Permitida", "Histórico conocido en T-0."),
40 }
41
42 tabla_leakage = pd.DataFrame(
43     [(col, 'info') for col, info in diagnostico_leakage.items()],
44     columns=["variable", "estado", "motivo"]
45 )
46 tabla_leakage["nunique_evidencia"] = tabla_leakage["variable"].map(df.nunique())
47
48 assert set(tabla_leakage["variable"]) == set(df.columns), "La tabla de leakage no cubre todas las columnas del dataset."
49 assert (tabla_leakage["estado"] == "Excluida").sum() == 4, "Se esperaban 4 columnas excluidas (3 constantes + 1 identificador)."
```

3. Preparación de los datos

La preparación se diseñó para que el procesamiento fuese reproducible y para evitar fuga de información durante el entrenamiento. Se separaron las necesidades de regresión y clasificación y se utilizaron pipelines de scikit-learn.

3.1 Limpieza de variables irrelevantes

Se eliminaron cuatro columnas. La evidencia cuantitativa mostró que EmployeeNumber tenía 1.470 valores únicos y que EmployeeCount, StandardHours y Over18 presentaban un solo valor. Después de la limpieza, el dataset quedó con 31 variables.

Evidencia cuantitativa para la eliminación de columnas

```

1  columnas_a_eliminar = ["EmployeeNumber", "EmployeeCount", "StandardHours", "Over18"]
2
3  evidencia = pd.DataFrame({
4      "nunique": df[columnas_a_eliminar].nunique(),
5      "varianza": [
6          df[c].var() if pd.api.types.is_numeric_dtype(df[c]) else np.nan
7          for c in columnas_a_eliminar
8      ],
9      "proporcion_valores_unicos": [df[c].nunique() / len(df) for c in columnas_a_eliminar],
10 })
11 print("Evidencia antes de eliminar:")
12 display(evidencia)
13
14 for col in ["EmployeeCount", "StandardHours", "Over18"]:
15     assert df[col].nunique() == 1, f"{col} no es constante, revisar antes de eliminar."
16     assert (not pd.api.types.is_numeric_dtype(df[col])) or (df[col].var() == 0), f"{col} no tiene varianza cero."
17 assert df["EmployeeNumber"].is_unique, "EmployeeNumber no es un identificador único."
18 assert evidencia.loc["EmployeeNumber", "proporcion_valores_unicos"] == 1.0, \
19     "EmployeeNumber debería tener un valor único por fila (proporción = 1.0)."
20
21 df_model = df.drop(columns=columnas_a_eliminar).copy()
22 print("\nColumnas eliminadas (constantes / identificador):", columnas_a_eliminar)
23 print("Forma tras limpieza:", df_model.shape)

```

3.2 Codificación del target de clasificación

Attrition se transforma en Attrition_flag, donde 1 representa fuga (Yes) y 0 representa permanencia (No). El conjunto mantiene 237 casos positivos.

Creación de Attrition_flag

```

1  df_model["Attrition_flag"] = df_model["Attrition"].map({"Yes": 1, "No": 0})
2
3  assert set(df_model["Attrition_flag"].unique()) == {0, 1}, "Attrition_flag debe ser binario 0/1."
4  assert df_model["Attrition_flag"].sum() == (df_model["Attrition"] == "Yes").sum(), "Mapeo inconsistente."
5  print("Attrition flag creado. Positivos (fuga):", int(df_model["Attrition_flag"].sum()),
6        "de", len(df_model))

```

3.3 Separación de variables por tarea

Para clasificación se utilizan las variables restantes incluyendo MonthlyIncome, porque el salario constituye información disponible antes de la fuga y puede aportar señal predictiva. Para regresión se excluyen MonthlyIncome y Attrition, evitando utilizar el propio target o el resultado de la otra tarea como predictor.

Separación de features y targets para las dos tareas

```

1 TARGET_REG = "MonthlyIncome"
2 TARGET_CLF_RAW = "Attrition"
3 TARGET_CLF = "Attrition_flag"
4
5 # Features para clasificación: todo menos las dos versiones del target de attrition
6 features_clf = [c for c in df_model.columns if c not in [TARGET_CLF_RAW, TARGET_CLF]]
7 X_clf = df_model[features_clf].copy()
8 y_clf = df_model[TARGET_CLF].copy()
9
10 # Features para regresión: lo anterior, menos el propio target de regresión
11 features_reg = [c for c in features_clf if c != TARGET_REG]
12 X_reg = df_model[features_reg].copy()
13 y_reg = df_model[TARGET_REG].copy()
14
15 print("X_reg:", X_reg.shape, "| X_clf:", X_clf.shape)

```

Tarea	Features	Target
Regresión	29 variables	MonthlyIncome
Clasificación	30 variables	Attrition_flag

3.4 Tratamiento de variables categóricas

Las variables categóricas nominales Department, EducationField, Gender, JobRole, MaritalStatus y OverTime se codifican mediante One-Hot Encoding. BusinessTravel se trata como ordinal, utilizando el orden Non-Travel < Travel_Rarely < Travel_Frequently. Las variables numéricas se imputan por mediana y se estandarizan con StandardScaler.

Identificación de variables numéricas, nominales y ordinales

```

1 ORDEN_ORDINALES = {
2     "BusinessTravel": ["Non-Travel", "Travel_Rarely", "Travel_Frequently"],
3 }
4
5 def columnas_por_tipo(X):
6     numericas = X.select_dtypes(include=[np.number]).columns.tolist()
7     categoricas = X.select_dtypes(include=["object", "category"]).columns.tolist()
8     ordinales = [c for c in categoricas if c in ORDEN_ORDINALES]
9     nominales = [c for c in categoricas if c not in ORDEN_ORDINALES]
10    return numericas, nominales, ordinales
11
12 num_cols_reg, catnom_cols_reg, catord_cols_reg = columnas_por_tipo(X_reg)
13 num_cols_clf, catnom_cols_clf, catord_cols_clf = columnas_por_tipo(X_clf)
14
15 print("Regresión -> numéricas:", len(num_cols_reg),
16       "| nominales:", len(catnom_cols_reg), "| ordinales:", len(catord_cols_reg))
17 print("Clasificación -> numéricas:", len(num_cols_clf),
18       "| nominales:", len(catnom_cols_clf), "| ordinales:", len(catord_cols_clf))
19 print("\nNominales:", catnom_cols_reg)
20 print("Ordinales (con orden explícito):", catord_cols_reg, "->", ORDEN_ORDINALES)

```

3.5 División entrenamiento/prueba

Se utilizó una partición 80/20. Para clasificación se aplicó estratificación para conservar la proporción de fugas en entrenamiento y prueba.

Tarea	Train	Test	Proporción positiva
Regresión	1.176 × 29	294 × 29	No aplica

Clasificación	1.176 × 30	294 × 30	Train 16,16% / Test 15,99%
---------------	------------	----------	----------------------------

Partición train/test y estratificación

```

1 X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
2     X_reg, y_reg, test_size=0.20, random_state=RANDOM_STATE
3 )
4
5 X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
6     X_clf, y_clf, test_size=0.20, random_state=RANDOM_STATE, stratify=y_clf
7 )
8
9 print("Regresión    -> train:", X_train_reg.shape, "| test:", X_test_reg.shape)
10 print("Clasificación -> train:", X_train_clf.shape, "| test:", X_test_clf.shape)
11 print("Proporción de fuga -> train:", round(y_train_clf.mean(), 4),
12       "| test:", round(y_test_clf.mean(), 4))

```

3.6 Pipelines reproducibles

El preprocesamiento se encapsula en ColumnTransformer y Pipeline. Esto permite que las transformaciones se ajusten únicamente con los datos de entrenamiento dentro de cada proceso de validación, reduciendo el riesgo de leakage. El preprocesador produce 48 variables transformadas en entrenamiento.

Construcción del preprocesador ColumnTransformer

```

1 def construir_preprocesador(columnas_numericas, columnas_nominales, columnas_ordinales):
2     pipe_num = Pipeline(steps=[
3         ("imputer", SimpleImputer(strategy="median")),
4         ("scaler", StandardScaler()),
5     ])
6
7     pipe_nominal = Pipeline(steps=[
8         ("imputer", SimpleImputer(strategy="most_frequent")),
9         ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
10    ])
11
12    pipe_ordinal = Pipeline(steps=[
13        ("imputer", SimpleImputer(strategy="most_frequent")),
14        ("ordinal", OrdinalEncoder(
15            categories=[ORDEN_ORDINALES[c] for c in columnas_ordinales],
16            handle_unknown="use_encoded_value", unknown_value=-1,
17        )),
18    ])
19
20    transformadores = [("num", pipe_num, columnas_numericas)]
21    if columnas_nominales:
22        transformadores.append(("nom", pipe_nominal, columnas_nominales))
23    if columnas_ordinales:
24        transformadores.append(("ord", pipe_ordinal, columnas_ordinales))
25
26    return ColumnTransformer(transformers=transformadores, remainder="drop")
27
28 # Chequeo rápido de que el preprocesador no produce nulos ni infinitos
29 _prep_chequeo = construir_preprocesador(num_cols_reg, catnom_cols_reg, catord_cols_reg)
30 _Xt_train = _prep_chequeo.fit_transform(X_train_reg)
31 _Xt_test = _prep_chequeo.transform(X_test_reg)
32
33 assert np.isfinite(_Xt_train).all(), "El preprocesador produjo valores no finitos en train."
34 assert np.isfinite(_Xt_test).all(), "El preprocesador produjo valores no finitos en test."
35 print("Preprocesador validado. Forma transformada (train):", _Xt_train.shape)

```

4. Modelado

La fase de modelado compara modelos simples con alternativas mejoradas. Se mantienen baselines porque permiten establecer una referencia mínima y demostrar si los modelos aprenden patrones útiles.

4.1 Modelado de regresión: MonthlyIncome

Baseline ingenuo

El baseline utiliza DummyRegressor con la estrategia de mediana. Su rendimiento fue MAE = 3.392,701, RMSE = 5.189,575 y $R^2 = -0,232$.

Baseline de regresión mediante DummyRegressor

```

1 baseline_reg = DummyRegressor(strategy="median")
2 baseline_reg.fit(X_train_reg, y_train_reg)
3 y_pred_dummy = baseline_reg.predict(X_test_reg)
4
5 def metricas_regresion(y_true, y_pred, nombre):
6     mae = mean_absolute_error(y_true, y_pred)
7     rmse = np.sqrt(mean_squared_error(y_true, y_pred))
8     r2 = r2_score(y_true, y_pred)
9     return {"modelo": nombre, "MAE": mae, "RMSE": rmse, "R2": r2}
10
11 resultados_reg = [metricas_regresion(y_test_reg, y_pred_dummy, "baseline_mediana")]
12 pd.DataFrame(resultados_reg).round(3)
```

Regresión Lineal

La Regresión Lineal se implementa dentro de un pipeline completo. En el conjunto de prueba alcanzó MAE = 895,372, RMSE = 1.169,208 y $R^2 = 0,937$, mostrando una mejora muy importante respecto al baseline.

Pipeline de Regresión Lineal

```

1 pipe_linear = Pipeline(steps=[
2     ("prep", construir_preprocesador(num_cols_reg, catnom_cols_reg, catord_cols_reg)),
3     ("model", LinearRegression()),
4 ])
5
6 pipe_linear.fit(X_train_reg, y_train_reg)
7 y_pred_linear = pipe_linear.predict(X_test_reg)
8
9 resultados_reg.append(metricas_regresion(y_test_reg, y_pred_linear, "regresion_lineal"))
10 pd.DataFrame(resultados_reg).round(3)
```

Ridge y Lasso

Debido a la colinealidad identificada en el EDA, se probaron Ridge y Lasso con GridSearchCV y cinco particiones de validación cruzada. El mejor alpha de Ridge fue 1,0 y el de Lasso 10,0.

Ajuste de hiperparámetros para Ridge y Lasso

```

1 cv_reg = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
2 alphas = [0.01, 0.1, 1.0, 10.0, 50.0, 100.0]
3
4 pipe_ridge = Pipeline(steps=[
5     ("prep", construir_preprocesador(num_cols_reg, catnom_cols_reg, catord_cols_reg)),
6     ("model", Ridge(random_state=RANDOM_STATE)),
7 ])
8 buscador_ridge = GridSearchCV(
9     pipe_ridge, param_grid={"model__alpha": alphas},
10    scoring="neg_root_mean_squared_error", cv=cv_reg, n_jobs=-1,
11 )
12 buscador_ridge.fit(X_train_reg, y_train_reg)
13
14 pipe_lasso = Pipeline(steps=[
15     ("prep", construir_preprocesador(num_cols_reg, catnom_cols_reg, catord_cols_reg)),
16     ("model", Lasso(random_state=RANDOM_STATE, max_iter=20000)),
17 ])
18 buscador_lasso = GridSearchCV(
19     pipe_lasso, param_grid={"model__alpha": alphas},
20    scoring="neg_root_mean_squared_error", cv=cv_reg, n_jobs=-1,
21 )
22 buscador_lasso.fit(X_train_reg, y_train_reg)
23
24 print("Mejor alpha Ridge:", buscador_ridge.best_params_, "| RMSE CV:", -buscador_ridge.best_score_)
25 print("Mejor alpha Lasso:", buscador_lasso.best_params_, "| RMSE CV:", -buscador_lasso.best_score_)
26
27 mejor_ridge = buscador_ridge.best_estimator_
28 mejor_lasso = buscador_lasso.best_estimator_
29
30 y_pred_ridge = mejor_ridge.predict(X_test_reg)
31 y_pred_lasso = mejor_lasso.predict(X_test_reg)
32
33 resultados_reg.append(metricas_regresion(y_test_reg, y_pred_ridge, "ridge_optimizado"))
34 resultados_reg.append(metricas_regresion(y_test_reg, y_pred_lasso, "lasso_optimizado"))
35
36 tabla_reg = pd.DataFrame(resultados_reg).round(3).sort_values("RMSE").reset_index(drop=True)
37 tabla_reg

```

Modelo	MAE	RMSE	R ²	Decisión
Regresión Lineal	895,372	1.169,208	0,937	Seleccionado
Ridge optimizado	896,385	1.171,329	0,937	Alternativa
Lasso optimizado	896,568	1.186,407	0,936	Alternativa
Baseline mediana	3.392,701	5.189,575	-0,232	Referencia

La Regresión Lineal fue seleccionada por obtener el menor RMSE en el conjunto de prueba. La validación cruzada del modelo seleccionado produjo RMSE medio de $1.148,40 \pm 73,38$.

Validación cruzada del modelo de regresión seleccionado

```

1 scores_cv_reg = cross_val_score(
2     modelo_reg_final, X_train_reg, y_train_reg,
3     scoring="neg_root_mean_squared_error", cv=cv_reg, n_jobs=-1,
4 )
5 print("RMSE por fold:", np.round(-scores_cv_reg, 2))
6 print(f"RMSE promedio (CV): {-scores_cv_reg.mean():.2f} +/- {-scores_cv_reg.std():.2f}")

```

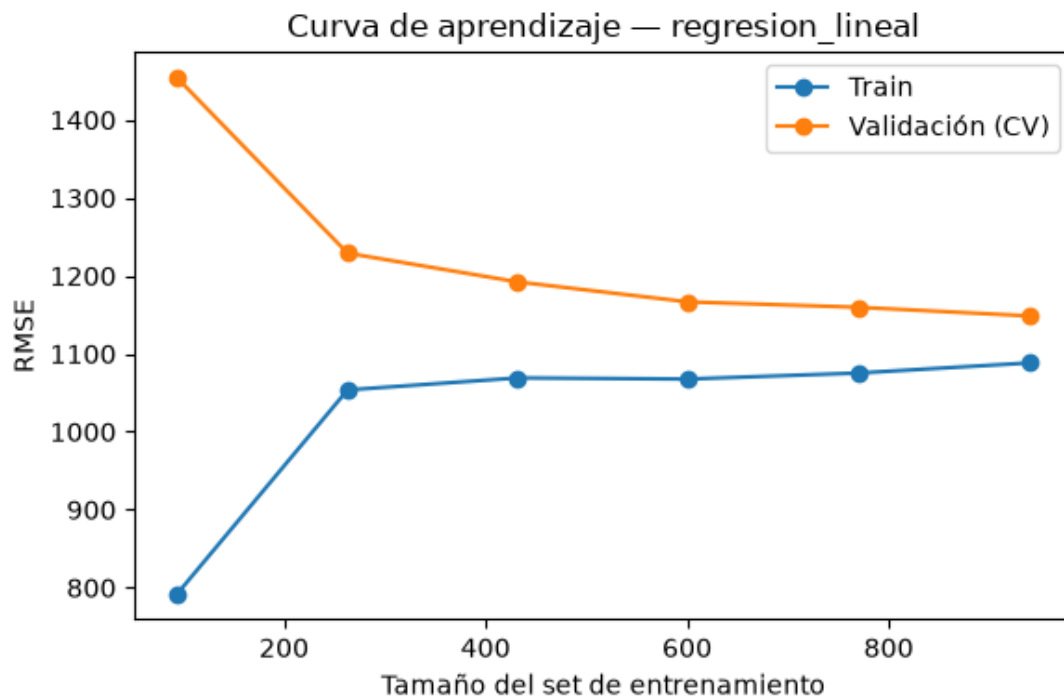


Figura 5. Curva de aprendizaje del modelo de regresión.

4.2 Modelado de clasificación: Attrition

Baseline ingenuo

El DummyClassifier que siempre predice No obtiene Accuracy = 0,840, pero Recall = 0 para la clase de fuga. Este resultado demuestra que Accuracy por sí sola sería engañosa.

Baseline de clasificación

```

1 baseline_clf = DummyClassifier(strategy="most_frequent", random_state=RANDOM_STATE)
2 baseline_clf.fit(X_train_clf, y_train_clf)
3 y_pred_dummy_clf = baseline_clf.predict(X_test_clf)
4 y_proba_dummy_clf = baseline_clf.predict_proba(X_test_clf)[: , 1]
5
6 print("Accuracy del baseline 'siempre No':", round((y_pred_dummy_clf == y_test_clf).mean(), 3))
7 print(classification_report(y_test_clf, y_pred_dummy_clf, digits=3, zero_division=0))

```

Regresión Logística balanceada

La Regresión Logística utiliza `class_weight='balanced'` para compensar el desbalance. El modelo base obtuvo ROC-AUC = 0,802 y PR-AUC = 0,555 en test.

Regresión Logística con balanceo de clases

```

1 pipe_logit = Pipeline(steps=[
2     ("prep", construir_preprocesador(num_cols_clf, catnom_cols_clf, catord_cols_clf)),
3     ("model", LogisticRegression(max_iter=2000, class_weight="balanced", random_state=RANDOM_STATE)),
4 ])
5
6 pipe_logit.fit(X_train_clf, y_train_clf)
7 y_pred_logit = pipe_logit.predict(X_test_clf)
8 y_proba_logit = pipe_logit.predict_proba(X_test_clf)[: , 1]
9
10 print(classification_report(y_test_clf, y_pred_logit, digits=3))
11 print("ROC-AUC:", round(roc_auc_score(y_test_clf, y_proba_logit), 3))
12 print("PR-AUC (average precision):", round(average_precision_score(y_test_clf, y_proba_logit), 3))

```

Ajuste de regularización

Se evaluaron distintos valores de C mediante StratifiedKFold y GridSearchCV, optimizando PR-AUC. El mejor C fue 0,05. En test, el modelo ajustado obtuvo ROC-AUC = 0,812 y PR-AUC = 0,589. La validación cruzada entregó ROC-AUC = $0,833 \pm 0,031$ y PR-AUC = $0,635 \pm 0,053$.

Búsqueda del hiperparámetro C

```

1 cv_clf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
2
3 pipe_logit_tune = Pipeline(steps=[
4     ("prep", construir_preprocesador(num_cols_clf, catnom_cols_clf, catorc_cols_clf)),
5     ("model", LogisticRegression(max_iter=2000, class_weight="balanced", random_state=RANDOM_STATE)),
6 ])
7
8 param_grid_clf = {"model__C": [0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0]}
9
10 buscador_logit = GridSearchCV(
11     pipe_logit_tune, param_grid=param_grid_clf,
12     scoring="average_precision", cv=cv_clf, n_jobs=-1,
13 )
14 buscador_logit.fit(X_train_clf, y_train_clf)
15
16 modelo_clf_final = buscador_logit.best_estimator_
17 print("Mejor C:", buscador_logit.best_params_, "| PR-AUC (CV):", round(buscador_logit.best_score_, 3))
18
19 y_pred_final = modelo_clf_final.predict(X_test_clf)
20 y_proba_final = modelo_clf_final.predict_proba(X_test_clf)[:, 1]
21
22 roc_auc_final = roc_auc_score(y_test_clf, y_proba_final)
23 pr_auc_final = average_precision_score(y_test_clf, y_proba_final)
24 print("\nEn test -> ROC-AUC:", round(roc_auc_final, 3), "| PR-AUC:", round(pr_auc_final, 3))

```

Modelo	ROC-AUC test	PR-AUC test	Observación
Baseline ingenuo	No representativo	0,160	No detecta fugas
Logística base	0,802	0,555	Mejora claramente al baseline
Logística ajustada	0,812	0,589	Modelo final de clasificación

Validación cruzada del modelo de clasificación

```

1 scores_cv_clf = cross_validate(
2     modelo_clf_final, X_train_clf, y_train_clf,
3     scoring={"roc_auc": "roc_auc", "pr_auc": "average_precision"},
4     cv=cv_clf, n_jobs=-1,
5 )
6 print(f"ROC-AUC (CV): {scores_cv_clf['test_roc_auc'].mean():.3f} +/- {scores_cv_clf['test_roc_auc'].std():.3f}")
7 print(f"PR-AUC (CV): {scores_cv_clf['test_pr_auc'].mean():.3f} +/- {scores_cv_clf['test_pr_auc'].std():.3f}")

```

5. Evaluación

La evaluación se realiza sobre el conjunto de prueba reservado, complementada con validación cruzada y diagnósticos visuales. Además de métricas estadísticas, se consideran las implicaciones de negocio de los errores.

5.1 Evaluación de la regresión

Tabla final de métricas de regresión

1	tabla_reg
---	-----------

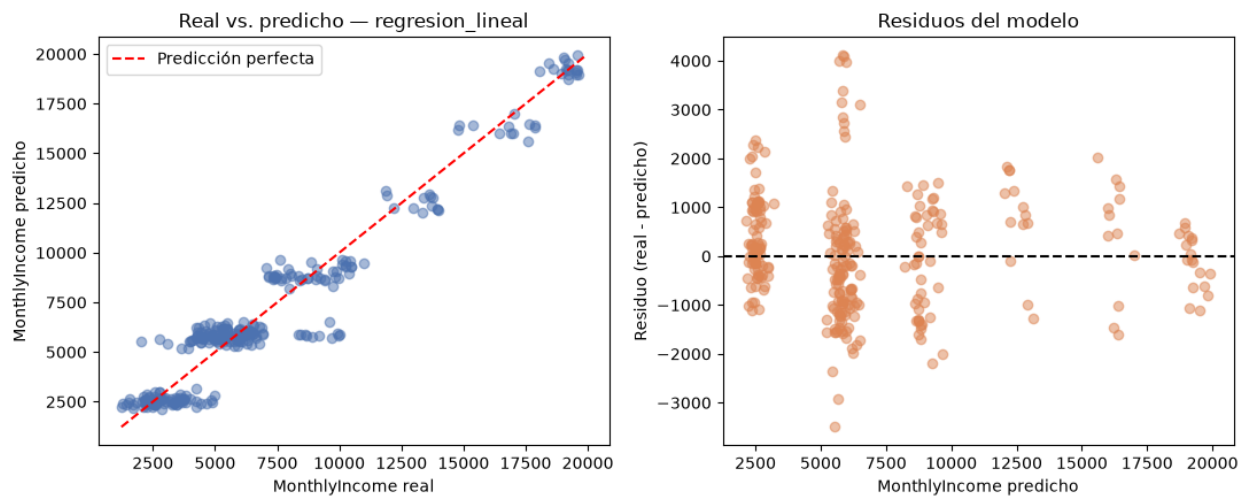


Figura 6. MonthlyIncome real frente a predicho y análisis de residuos.

Los resultados indican un ajuste fuerte: $R^2 = 0,937$ significa que el modelo explica una parte muy elevada de la variabilidad observada en MonthlyIncome dentro del conjunto de prueba. El RMSE de 1.169,208 debe interpretarse como un error típico penalizado por errores grandes; el MAE de 895,372 ofrece una lectura más directa del error absoluto medio.

5.2 Evaluación de la clasificación

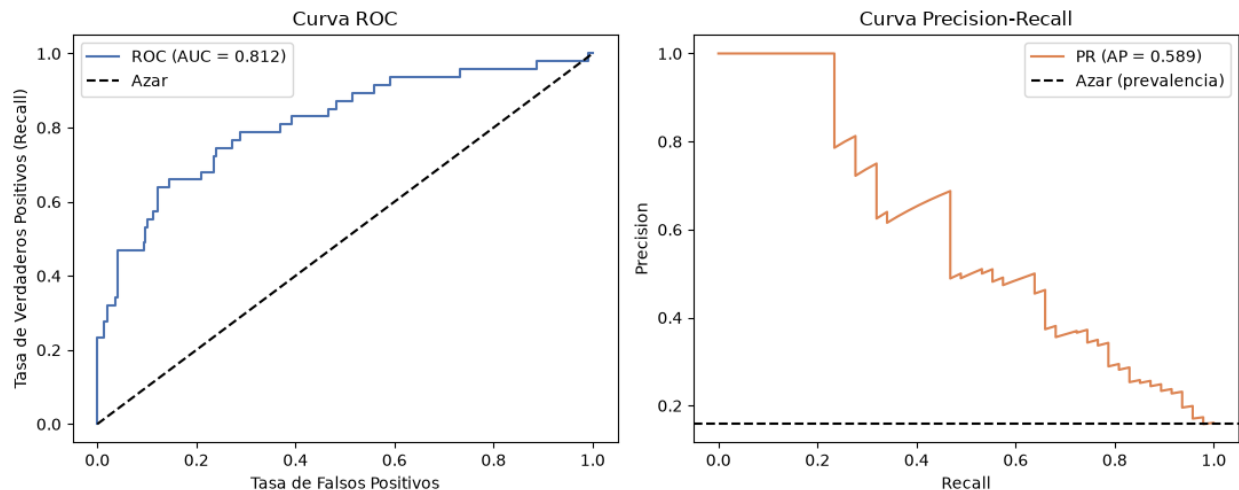


Figura 7. Curvas ROC y Precision-Recall del modelo final.

La curva ROC permite evaluar la capacidad de discriminación global, mientras que Precision-Recall resulta especialmente útil ante el desbalance de clases. El PR-AUC de 0,589 en test representa una mejora sustancial frente al baseline ingenuo de 0,160.

5.3 Selección del umbral operativo

El notebook no asume que el umbral 0,50 sea óptimo. Se define un costo relativo de 5 para cada falso negativo y 1 para cada falso positivo, reflejando que perder talento se considera más costoso que realizar una intervención preventiva innecesaria. Al evaluar umbrales entre 0,05 y 0,95, el mínimo costo observado corresponde a 0,55.

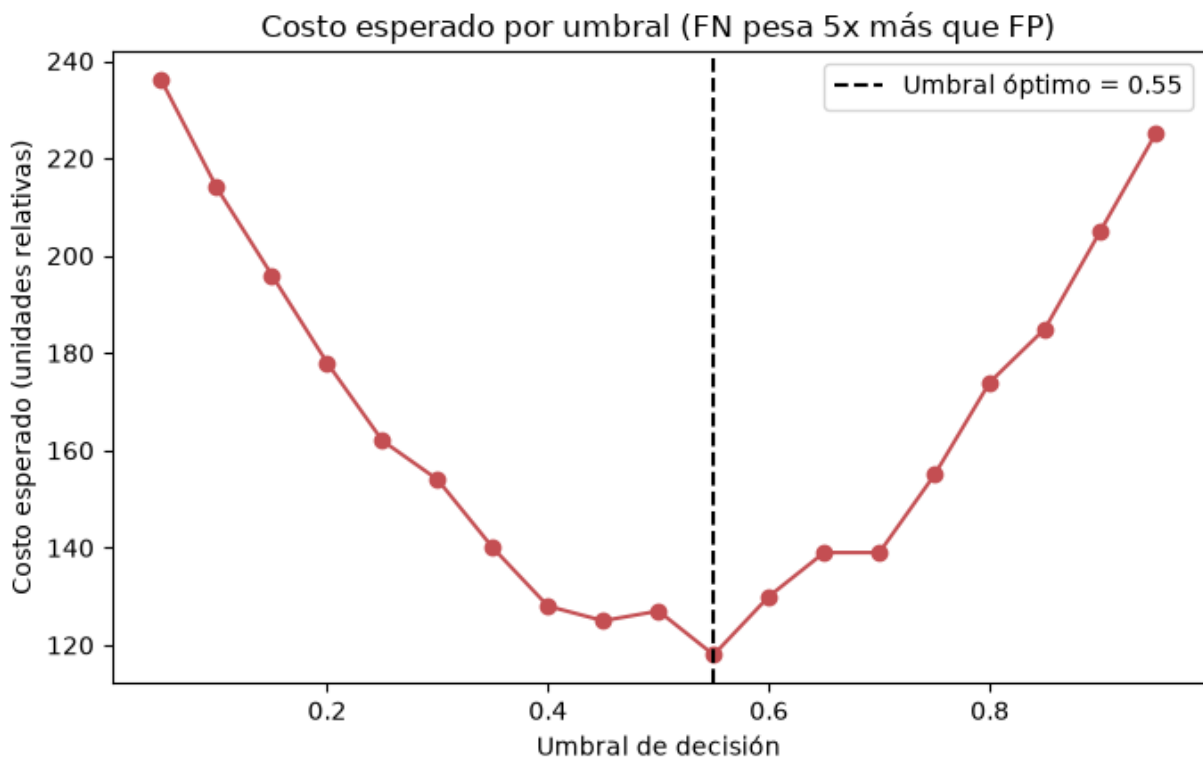


Figura 8. Evaluación del costo esperado según el umbral.

Cálculo del costo de falsos negativos y falsos positivos

```

1 COSTO_FN = 5.0 # relativo: perder una fuga no detectada
2 COSTO_FP = 1.0 # relativo: intervención de retención innecesaria
3
4 umbrales_prueba = np.linspace(0.05, 0.95, 19)
5 resultados_umbral = []
6
7 for u in umbrales_prueba:
8     y_pred_u = (y_proba_final >= u).astype(int)
9     tn, fp, fn, tp = confusion_matrix(y_test_clf, y_pred_u).ravel()
10    costo = COSTO_FN * fn + COSTO_FP * fp
11    precision_u = tp / (tp + fp) if (tp + fp) > 0 else 0.0
12    recall_u = tp / (tp + fn) if (tp + fn) > 0 else 0.0
13    resultados_umbral.append({
14        "umbral": u, "FN": fn, "FP": fp, "costo": costo,
15        "precision": precision_u, "recall": recall_u,
16    })
17
18 tabla_umbral = pd.DataFrame(resultados_umbral)
19 umbral_optimo = tabla_umbral.loc[tabla_umbral["costo"].idxmin(), "umbral"]
20
21 plt.figure(figsize=(7, 4.5))
22 plt.plot(tabla_umbral["umbral"], tabla_umbral["costo"], "o-", color="#C44E52")
23 plt.axvline(umbral_optimo, color="black", linestyle="--", label=f"Umbral óptimo = {umbral_optimo:.2f}")
24 plt.xlabel("Umbral de decisión")
25 plt.ylabel("Costo esperado (unidades relativas)")
26 plt.title("Costo esperado por umbral (FN pesa 5x más que FP)")
27 plt.legend()
28 plt.tight_layout()
29 plt.show()
30
31 print(f"Umbral operativo elegido: {umbral_optimo:.2f}")
32 tabla_umbral.round(3)

```

Métrica	Umbral 0,50	Umbral 0,55
Precision clase fuga	0,381	0,449
Recall clase fuga	0,681	0,660
F1 clase fuga	0,489	0,534
Falsos negativos	15	16
Falsos positivos	52	38
Costo relativo	127	118

Con el umbral 0,55 se reducen los falsos positivos y el costo relativo total, aunque el Recall baja ligeramente de 0,681 a 0,660. La matriz de confusión resultante es [[209, 38], [16, 31]]. La elección se justifica por el costo esperado y no únicamente por Accuracy.

Comparación de desempeño con umbral 0,50 y umbral operativo 0,55

```

1 y_pred_operativo = (y_proba_final >= umbral_optimo).astype(int)
2
3 print(f"--- Umbral por defecto (0.50) ---")
4 print(classification_report(y_test_clf, y_pred_final, digits=3))
5
6 print(f"\n--- Umbral operativo ({umbral_optimo:.2f}) ---")
7 print(classification_report(y_test_clf, y_pred_operativo, digits=3))
8
9 print("\nMatriz de confusión (umbral operativo):\n",
10       confusion_matrix(y_test_clf, y_pred_operativo))

```

5.4 Criterios de aceptación del modelado

Control	Resultado
Modelos formales superan al baseline de regresión	Sí
Estratificación de clasificación verificada	Sí
Modelo de clasificación supera baseline en PR-AUC	Sí
Validación cruzada de regresión	Realizada
Validación cruzada de clasificación	Realizada
Diagnóstico de learning curve	Realizado
Análisis de umbral por costo	Realizado

6. Interpretación de resultados

6.1 Regresión: efecto de la regularización

El notebook compara los coeficientes de Regresión Lineal, Ridge y Lasso. Ridge reduce la magnitud de coeficientes asociados a variables colineales sin eliminarlos, mientras que Lasso puede llevar coeficientes exactamente a cero. En este caso, Lasso llevó 21 de 48 coeficientes a cero, proporcionando una representación más escasa.

Comparación de coeficientes entre Regresión Lineal, Ridge y Lasso

```

1 nombres_features = pipe_linear.named_steps["prep"].get_feature_names_out()
2
3 tabla_comparacion_coef = pd.DataFrame({
4     "variable": nombres_features,
5     "lineal": pipe_linear.named_steps["model"].coef_,
6     "ridge": mejor_ridge.named_steps["model"].coef_,
7     "lasso": mejor_lasso.named_steps["model"].coef_,
8 })
9 tabla_comparacion_coef["abs_lineal"] = tabla_comparacion_coef["lineal"].abs()
10 tabla_comparacion_coef = tabla_comparacion_coef.sort_values("abs_lineal", ascending=False)
11
12 n_ceros_lasso = int((tabla_comparacion_coef["lasso"] == 0).sum())
13 print(f"Lasso llevó exactamente a 0 a {n_ceros_lasso} de {len(tabla_comparacion_coef)} coeficientes "
14       "(selección automática de variables).")
15 print(f"Norma L2 de los coeficientes -> lineal: {np.linalg.norm(tabla_comparacion_coef['lineal']):.1f} | "
16       f"ridge: {np.linalg.norm(tabla_comparacion_coef['ridge']):.1f} | "
17       f"lasso: {np.linalg.norm(tabla_comparacion_coef['lasso']):.1f}")
18
19 top12 = tabla_comparacion_coef.head(12)
20 x = np.arange(len(top12))
21 ancho = 0.25
22
23 plt.figure(figsize=(9, 6))
24 plt.barh(x - ancho, top12["lineal"][::-1], height=ancho, label="Lineal (sin regularizar)")
25 plt.barh(x, top12["ridge"][::-1], height=ancho, label="Ridge")
26 plt.barh(x + ancho, top12["lasso"][::-1], height=ancho, label="Lasso")
27 plt.yticks(x, top12["variable"][::-1])
28 plt.axvline(0, color="black", linewidth=0.8)
29 plt.xlabel("Coeficiente (variables estandarizadas)")
30 plt.title("Encogimiento de coeficientes: Lineal vs. Ridge vs. Lasso")
31 plt.legend()
32 plt.tight_layout()
33 plt.show()
34
35 tabla_comparacion_coef[["variable", "lineal", "ridge", "lasso"]].head(15)

```

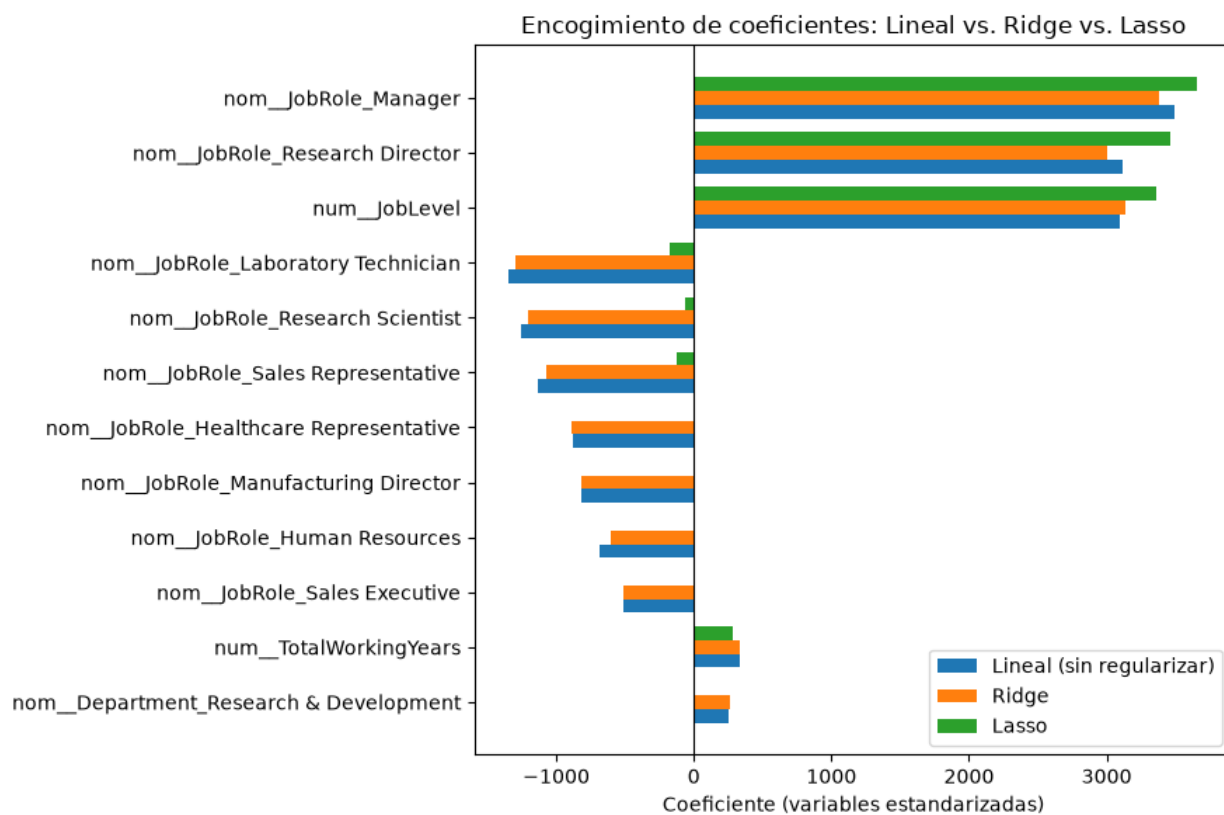


Figura 9. Comparación visual de coeficientes de los modelos de regresión.

6.2 Variables que explican MonthlyIncome

En el modelo seleccionado, las variables con mayor peso incluyen JobRole, JobLevel y TotalWorkingYears. El signo y magnitud de los coeficientes deben interpretarse dentro del espacio estandarizado y codificado del pipeline, por lo que no equivalen directamente a un aumento absoluto en unidades monetarias por cada unidad de una variable original.

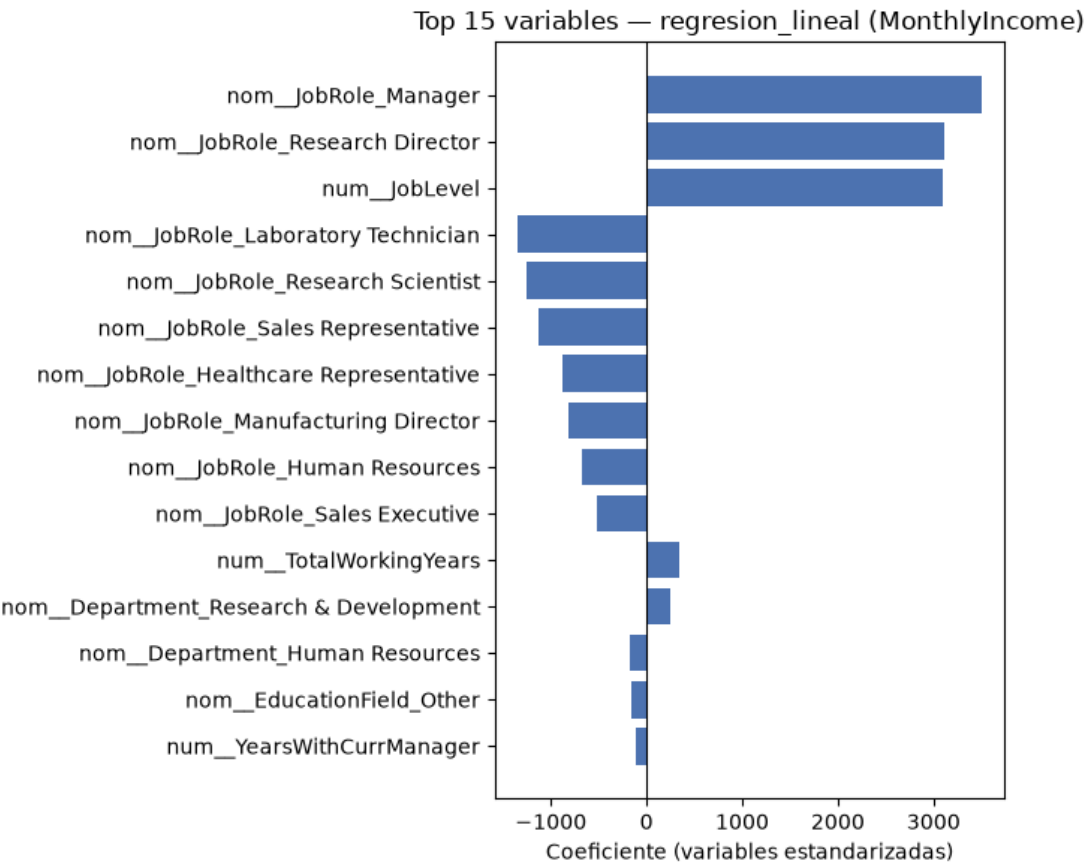


Figura 10. Principales coeficientes del modelo seleccionado de salario.

6.3 Variables sensibles y consideraciones éticas

Se revisó explícitamente el peso de Gender, MaritalStatus y Age. En el modelo salarial, los coeficientes observados son pequeños frente a los asociados con JobRole, JobLevel y antigüedad. Sin embargo, esto no demuestra por sí mismo ausencia de sesgo. Se recomienda auditar periódicamente diferencias de predicción y residuos entre grupos.

Variable	Coefficiente reportado
Age	-36,007
Gender_Male	23,294
Gender_Female	-23,294
MaritalStatus_Single	-15,806
MaritalStatus_Married	15,358
MaritalStatus_Divorced	0,448

6.4 Clasificación: factores asociados al riesgo de fuga

La interpretación de la Regresión Logística utiliza odds ratios. OverTime_Yes presenta un odds ratio aproximado de 1,829, lo que indica una asociación positiva con la probabilidad estimada de fuga. También aparecen BusinessTravel y determinados roles con asociaciones positivas. Por otra parte, mayor satisfacción laboral, antigüedad y MonthlyIncome aparecen con asociaciones negativas en el modelo.

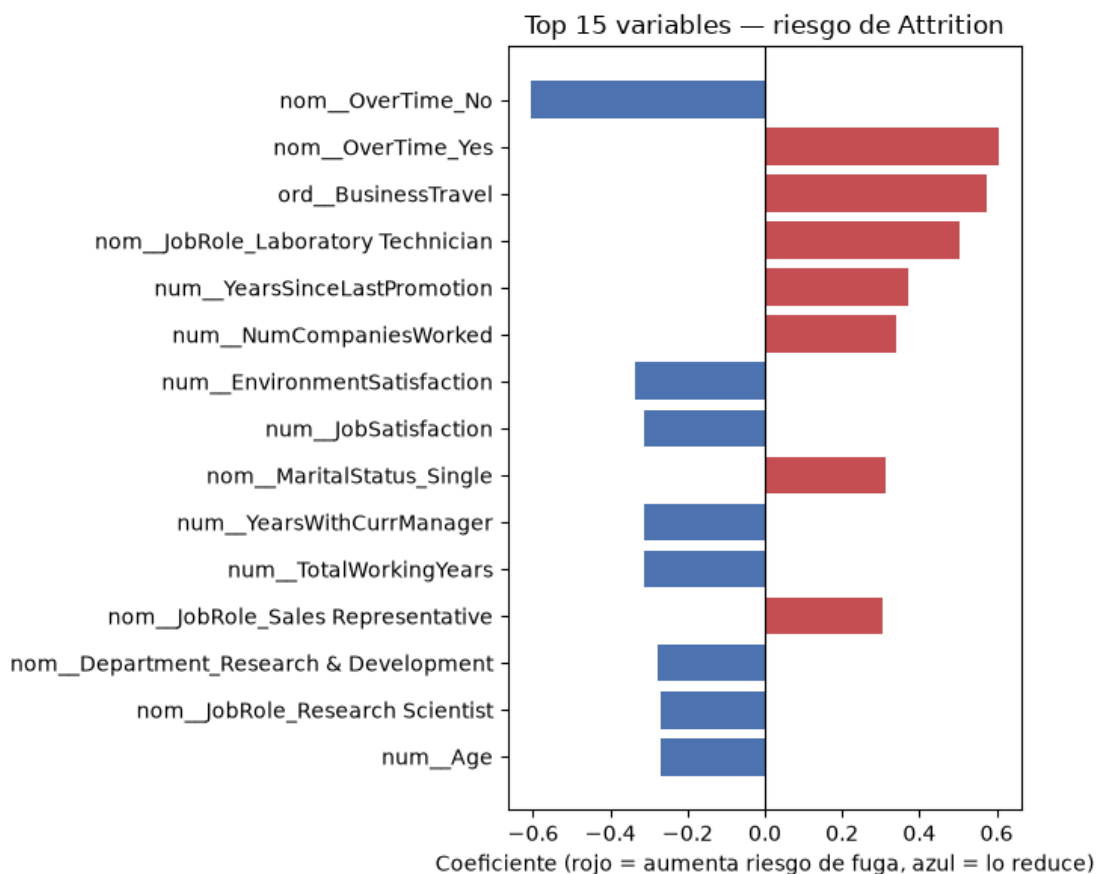


Figura 11. Principales coeficientes/odds ratios del modelo de Attrition.

6.5 Uso responsable del modelo

- El modelo debe utilizarse como apoyo para conversaciones de retención, no como mecanismo de decisión automática.
- Las variables demográficas sensibles no deberían emplearse para negar beneficios, oportunidades o tratamientos laborales.
- Una asociación estadística no demuestra causalidad.
- Las alertas deben revisarse con contexto humano antes de ejecutar cualquier intervención.

7. Recomendaciones de negocio

7.1 Compensación y equidad salarial

1. Priorizar la revisión de empleados cuyos residuos sean muy negativos, es decir, cuyo salario real esté considerablemente por debajo del salario estimado por el modelo para perfiles comparables.
2. Utilizar el modelo como herramienta de auditoría y diagnóstico, no como determinante automático de aumentos.
3. Revisar especialmente las bandas salariales asociadas con nivel jerárquico, rol y antigüedad.
4. Realizar auditorías periódicas por género, edad y estado civil para detectar posibles patrones de inequidad.

7.2 Retención de talento

5. Priorizar empleados con probabilidad estimada alta de Attrition utilizando el umbral operativo de 0,55.
6. Revisar carga laboral y uso de horas extra, debido a la fuerte asociación observada entre OverTime y fuga.
7. Implementar acciones de bienestar y satisfacción laboral en los segmentos con mayor riesgo.
8. Dar prioridad a empleados cuya salida pueda representar una pérdida importante de conocimiento organizacional.
9. Registrar el resultado de las intervenciones para poder medir posteriormente si las acciones de retención fueron efectivas.

7.3 Limitaciones

- El dataset representa una fotografía histórica y corresponde a una única organización; por ello, la generalización hacia otras empresas es limitada.
- Correlación no implica causalidad.
- Los costos $FN = 5$ y $FP = 1$ son supuestos ilustrativos y deberían calibrarse con costos reales de reemplazo.
- La clasificación tiene una clase positiva relativamente pequeña, por lo que el desempeño debe monitorearse con especial atención.
- No se realiza despliegue en esta etapa y, por tanto, no se han evaluado drift, latencia ni comportamiento con datos productivos nuevos.

7.4 Próximos pasos

- Validar las recomendaciones con RRHH y Finanzas.
- Calibrar los costos de falsos positivos y falsos negativos con información real.
- Enriquecer el dataset con encuestas de clima, entrevistas de salida y variables temporales.
- Evaluar técnicas adicionales para desbalance y calibración de probabilidades.
- Monitorear desempeño y drift cuando exista un escenario de operación real.

8. Conclusiones

La aplicación de CRISP-DM permitió estructurar el proyecto desde una necesidad de negocio concreta hasta una evaluación cuantitativa de modelos. La fase de Comprensión de los Datos mostró que el dataset posee buena calidad inicial, pero presenta dos características importantes: una distribución sesgada de MonthlyIncome y un fuerte desbalance de Attrition.

En regresión, la Regresión Lineal obtuvo el mejor desempeño de los modelos comparados, con $R^2 = 0,937$ y $RMSE = 1.169,208$ en test. Ridge y Lasso demostraron ser alternativas útiles para tratar la colinealidad y simplificar coeficientes, aunque no superaron al modelo lineal según el RMSE final.

En clasificación, el baseline ingenuo confirmó que Accuracy no es suficiente: un 84% de Accuracy podía esconder un Recall de cero para las fugas. La Regresión Logística balanceada y posteriormente ajustada

alcanzó PR-AUC = 0,589 en test y PR-AUC = $0,635 \pm 0,053$ en validación cruzada. La selección de umbral 0,55 permitió traducir el problema estadístico a una decisión de negocio basada en costos.

Finalmente, los resultados sugieren que JobLevel, JobRole y antigüedad son factores relevantes para explicar MonthlyIncome, mientras que OverTime, satisfacción laboral, antigüedad y otras características aparecen asociadas al riesgo estimado de Attrition. Estas conclusiones deben utilizarse como apoyo para decisiones humanas y acompañarse de controles de equidad y validación organizacional.

Anexo A. Resumen técnico del notebook

Elemento	Implementación
Lenguaje	Python
EDA	Pandas + Matplotlib
Preprocesamiento	ColumnTransformer + Pipeline
Variables nominales	OneHotEncoder
Variable ordinal	OrdinalEncoder para BusinessTravel
Escalado	StandardScaler
Imputación	Mediana / moda
Regresión	DummyRegressor, LinearRegression, Ridge, Lasso
Clasificación	DummyClassifier, LogisticRegression
Búsqueda de hiperparámetros	GridSearchCV
CV regresión	KFold, 5 folds
CV clasificación	StratifiedKFold, 5 folds
Métricas regresión	MAE, RMSE, R ²
Métricas clasificación	Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC
Diagnóstico	Learning curve, residuos, ROC, Precision-Recall, matriz de confusión
Umbral operativo	Costo FN = 5, FP = 1; umbral seleccionado 0,55

Anexo B. Evidencia de ejecución y controles

Control	Evidencia
Dataset	1.470 × 35; targets presentes
Nulos	0 valores nulos
Limpieza	4 columnas eliminadas; 1.470 × 31
Splits	1.176 train / 294 test
Transformación	48 variables transformadas en train
Regresión	Todos los modelos formales superan el baseline
Clasificación	PR-AUC final 0,589 > baseline 0,160
Validación	CV realizada para ambas tareas
Leakage	Revisión explícita de las 35 variables
Ética	Variables sensibles revisadas y recomendaciones de uso responsable

Nota: El presente informe no incluye una fase de despliegue, porque el notebook analizado se encuentra en una etapa de desarrollo, modelado y evaluación. La sección de próximos pasos indica qué sería necesario considerar antes de llevar los modelos a un entorno operativo.